



Security Audit

USDU Core

16. - 24.07.2025



Table of Contents

Introduction.....	3
Overview.....	3
Scope.....	3
Functionality.....	3
Authorities.....	4
Findings.....	6
[LOW-1] Inconsistency between _setDistribution() and _distribute().....	6
[LOW-2] Incorrect totalWeights calculation.....	8
[LOW-3] Accidental token deposits are exclusively distributed to the last receiver.....	10
[LOW-4] Invalid token index check in CurveAdapterV1 constructor Accidental token deposits are exclusively distributed to the last receiver.....	11
[NIT-1] Unnecessary call to approve() in MorphoAdapterV1.redeem().....	13
Techniques.....	15
Disclaimer.....	16

Introduction

USDU engaged [BlockBite](#) to do a detailed security analysis of their new USDU stable coin smart contracts.

The audit revealed 4 low-severity issues; all of them are resolved or accepted by the project.

The following report describes all findings in details.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities complement but don't replace other vital measures to secure a project. The following sections will give an overview of the system, the issues uncovered and how they have been addressed.

Overview

Scope

The contract audit's scope comprised of three major components:

- Primarily, the Implementation security of the contract's source code
- Additionally, security of the overall design
- Additionally, resilience against economical attacks

The following contracts in the [USDU-Core](#) repository were considered to be in scope:

- contracts/stablecoin/Stablecoin.sol
- contracts/morpho/MorphoAdapterV1.sol
- contracts/curve/CurveAdapterV1.sol

Third-party dependencies or any other source files in the repository are not in scope.

The relevant commits are as following:

- [bf78b7dc8e148f0b805980f7b9e65a4e5d738d50](#) – Start of the audit
- [a8a24666e03e77306c92793f7b73d5bb2b7af0b4](#) – Last reviewed revision

Functionality

USDU is a new stable coin pegged to the US-Dollar. The peg is achieved by binding USDU to other US-Dollar stable coins via Morpho Vaults and Curve Stableswap pools.

The protocol consists of the core ERC-20 contract and modules for the protocols that the token integrates with.

The Stablecoin contract allows modules to be registered via the curator or from anyone by paying a public fee in USDU. Proposed modules are then subject to a timelock in which either the curator or the guardian can veto the proposal. Any paid fees are not returned in case of a veto.

The token contract also includes a freeze/unfreeze function to prevent a user from sending any tokens in case their balance is frozen.

The modules are responsible for keeping the vaults/pools in balance so that the peg USDU → US-Dollar is held at a steady 1:1.

The MorphoAdapter contract does this by depositing or redeeming USDU from the Morpho Vault. This action is permissioned and can only be executed by the protocol's curator. The contract keeps track of any debt it currently has to the protocol, which can be reduced by redeeming USDU from the Morpho market. Any left over profits will be distributed to up to 5 receivers unique to the module.

Any Vault-Tokens received during deposits are redeposited into another Morpho Vault.

The CurveAdapter follows a similar pattern but allows any user to deposit or remove liquidity to the Curve Stableswap pool. To add liquidity a user only needs to deposit the token that the pool is paired with, the module mints any needed USDU freshly from the protocol. The resulting LP-Tokens are split evenly between the user and the module.

Redeeming works in the same way: the module doubles any LP-Tokens the user wants to redeem and splits the proceeds evenly between the user and the protocol, in turn reducing the debt the module has in the protocol.

Both actions are limited by the balance of the pool. For providing liquidity the module requires that the pool has a deficit in USDU after adding liquidity, while during redemption the module requires the pool to not end in a deficit of USDU after the withdrawal.

Any change in the functionality of the protocol, such as curator, guardian or modules, are bound to a timelock in which the curator or guardian (depending on the action) can veto the change.

Authorities

As seen in the functionality section the protocol has two authorities present. At the time of writing the curator role is set to a [Aragon](#) DAO while the guardian is not set yet. It is planned for the future that sUSDU (Vault-Tokens from the first Morpho Vault) will be used as guardian tokens. These guardian will receive voting power based on the amount of sUSDU they hold and time that has passed.

Curator

The curator role can perform various permissioned actions in the token contract itself but also in the modules.

In the token contract the curator can propose a new curator, guardian or module. It can also veto pending curators, guardians, modules or change the timelock required for such a change. Lastly it is allowed to freeze any balance immediately.

In the modules the curator can request a change to the proceed receivers (Both modules), redeem LP-Tokens from the module only (Curve), run the reconciliation (Curve), deposit or redeem USDU (Morpho Vault).

Guardian

The guardian role is the oversight for the curator. It can veto any change proposed by the public or the curator.

In addition to these power the guardian can also propose the unfreezing of an accounts balance in the token contract or trigger a reconciliation in the Curve Module.

Findings

[LOW-1] Inconsistency between `_setDistribution()` and `_distribute()`

Description

There is an inconsistency between how `receivers` and `totalWeights` are set compared to how profits are distributed.

The inconsistency comes from the early function exit on [L237](#) if `receiver == address(0)`.

This inconsistency results in 2 issues:

Issue 1

Not all receivers get their profit share as expected.

Assuming `receivers[]` has the following entries: `[0xdead...beef, 0x1234...5678, 0x0000...0000, 0xabcd...ef90, 0x0000...0000]`.

Even though the receiver `0xabcd...ef90` is listed, they will never receive a payout of the profits.

Issue 2

The early exit also causes the `totalWeight` calculation not to reflect the actual weight of a recipient.

Assuming the same setup for `receivers[]` paired with the following weights `[25, 25, 0, 50, 0]` and `totalWeights = 100`.

For `0xdead...beef`, the function calculates their split as expected to be $1/4$ of the total profits.

For `0x1234...5678`, the function will set the split to the remaining balance as see on lines 240-243.

This results in the following payouts:

`0xdead...beef` = $1/4$

`0x1234...5678` = $3/4$

`0xabcd...ef90` = 0

Source Code

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/morpho/MorphoAdapterV1.sol#L124-L141>

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/morpho/MorphoAdapterV1.sol#L230-L254>

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/curve/CurveAdapterV1.sol#L120-L137>

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/curve/CurveAdapterV1.sol#L285-L309>

```
124 function _setDistribution(address[5] memory _receivers, uint32[5] memory _weights) internal {
125     // reset totalWeights
126     totalWeights = 0;
127
128     // update total weight
129     for (uint32 i = 0; i < _receivers.length; i++) {
130         totalWeights += _weights[i];
131     }
132
133     // update distribution
134     receivers = _receivers;
135     weights = _weights;
136
137     // emit event
138     emit SetDistribution(_msgSender());
139
140     _cleanUpPending();
141 }
```

```
230 function _distribute(uint256 amount) internal {
231     for (uint256 i = 0; i < 5; i++) {
232         address receiver = receivers[i];
233         uint256 weight = weights[i];
234         uint256 split;
235
236         // end distribution
237         if (receiver == address(0)) return;
238
239         // last item reached (index: 5 - 1 = 4) OR next receiver is zeroAddress
240         if (i == 4 || (i < 4 && receivers[i + 1] == address(0))) {
241             // distribute remainings, eliminating rounding or deposit issues
242             split = stable.balanceOf(address(this));
243         } else {
244             // distribute weighted split
245             split = (weight * amount) / totalWeights;
246         }
247
248         // distribute revenue split
249         stable.transfer(receiver, split);
250
251         // last weighted ratio might be inconsistent, due to remaining assets distribution
252         emit Distribution(receiver, split, (weight * 1 ether) / totalWeights);
253     }
254 }
```

Suggestion

Add a check to the public `setDistribution()` function to verify that there are not trailing entries after the first `0x0000...0000`.

An additional recommendation is to switch `setDistribution()` from external to public and use it in the constructor, instead of the internal `_setDistribution()`.

Resolution

Inconsistency is known and relies on correct data input and sequence.

[LOW-2] Incorrect totalWeights calculation

Description

`totalWeights` is calculated to be the sum of `weights[]` given to `_setDistribution()`. This calculation does not include a check for `0x0000...0000`, which results in a skewed `totalWeights` amount for the benefit of the last receiver set in `receiver[]`. (See LOW-1 to know who is the last receiver.)

Source Code

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/morpho/MorphoAdapterV1.sol#L124-L141>

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/curve/CurveAdapterV1.sol#L120-L137>

```
124 function _setDistribution(address[5] memory _receivers, uint32[5] memory _weights) internal {
125     // reset totalWeights
126     totalWeights = 0;
127
128     // update total weight
129     for (uint32 i = 0; i < _receivers.length; i++) {
130         totalWeights += _weights[i];
131     }
132
133     // update distribution
134     receivers = _receivers;
135     weights = _weights;
136
137     // emit event
138     emit SetDistribution(_msgSender());
139
140     _cleanUpPending();
141 }
```

Suggestion

Add a check in the for loop `receivers[I] != address(0)` to only count weights that are set for valid addresses.

Resolution

Inconsistency is known and relies on correct data input and sequence. Also it would allow to keep a portion in the contract.

Suggestion applied to `RewardDistributionV1` abstract contract, which isolates the distribution logic and could apply in future modules and updates.

The `Reward DistributorV1` contract was introduced with the commits [4f072048ddbe5957f3f3925de941a14bb00811c0](#) (Morpho) and [68f0d43eede1ea31331d9171efa4118ed1b84833](#) (Curve).

[LOW-3] Accidental token deposits are exclusively distributed to the last receiver

Description

The function `_distribute(amount)` takes in an amount which is then shared between the receivers. This is true for all receivers except the last one, who gets the remaining balance of USDU of the contract.

While this is a great approach to clear dust or rounding errors, it leads to an imbalanced distribution of accidentally deposited tokens.

Source Code

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/morpho/MorphoAdapterV1.sol#L230-L254>

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/curve/CurveAdapterV1.sol#L285-L309>

```
230 function _distribute(uint256 amount) internal {
231     for (uint256 i = 0; i < 5; i++) {
232         address receiver = receivers[i];
233         uint256 weight = weights[i];
234         uint256 split;
235
236         // end distribution
237         if (receiver == address(0)) return;
238
239         // last item reached (index: 5 - 1 = 4) OR next receiver is zeroAddress
240         if (i == 4 || (i < 4 && receivers[i + 1] == address(0))) {
241             // distribute remainings, eliminating rounding or deposit issues
242             split = stable.balanceOf(address(this));
243         } else {
244             // distribute weighted split
245             split = (weight * amount) / totalWeights;
246         }
247
248         // distribute revenue split
249         stable.transfer(receiver, split);
250
251         // last weighted ratio might be inconsistent, due to remaining assets distribution
252         emit Distribution(receiver, split, (weight * 1 ether) / totalWeights);
253     }
254 }
```

Suggestion

The contract is designed to maintain a 0 USDU balance during regular operation, so distributing the entire current balance is the a valid behavior for revenue distribution.

Remove the amount as a parameter from `_distribute()` and always distribute the current balance.

Resolution

The suggestion was implemented with [39d7d9a7e88428c0f4b200616ca3fe9b45091221](https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/curve/CurveAdapterV1.sol#L162-L194) by distributing the total balance instead of the given amount.

[LOW-4] Invalid token index check in CurveAdapterV1 constructor Accidental token deposits are exclusively distributed to the last receiver

Description

The constructor of CurveAdapterV1 checks if the indices for Stable (`idxS`) and the pairing coin (`idxC`) is within the bounds of max tokens.

While a Curve StableSwapNG pool supports up to 8 tokens, the CurveAdapterV1 does not.

`addLiquidity()` creates an `amounts[]` array with a length of 2, which results in a revert if one of the indices is > 1 .

Source Code

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/curve/CurveAdapterV1.sol#L162-L194>

```
162 function addLiquidity(uint256 amount, uint256 minShares) external returns (uint256) {
163     uint256 amountStable = (amount * 1 ether) / 10 ** coin.decimals();
164
165     // transfer coin token, needs approval
166     coin.safeTransferFrom(_msgSender(), address(this), amount);
167
168     // mint the same amountStable in stables
169     stable.mintModule(address(this), amountStable);
170     totalMinted += amountStable;
171
172     // approve tokens
173     stable.forceApprove(address(pool), amountStable);
174     coin.forceApprove(address(pool), amount);
175
176     // prepare amounts
177     uint256[] memory amounts = new uint256[](2);
178     amounts[idxS] = amountStable;
179     amounts[idxC] = amount;
180
181     // provide liquidity
182     uint256 shares = pool.add_liquidity(amounts, minShares * 2);
183
184     // verify imbalance for stable
185     verifyImbalance(true);
186
187     // return sender's split of shares
188     uint256 split = shares / 2;
189     pool.transfer(_msgSender(), split);
190
191     // emit event and return share split
192     emit AddLiquidity(_msgSender(), amountStable, totalMinted, split, pool.balanceOf(address(this)));
193     return split;
194 }
```

Suggestion

There are 2 ways to resolve this issue:

1. Change the requirement in the constructor for an index to be ≤ 1
2. Increase the length of the amounts array in `addLiquidity()` to be 8.

With solution #2, the team should keep in mind that `checkImbalance()` only checks for an imbalance between the two tokens involved, not that the whole pool is balanced with all involved assets.

Resolution

The issue was fixed commit [a8a24666e03e77306c92793f7b73d5bb2b7af0b4](#).

[NIT-1] Unnecessary call to approve() in MorphoAdapterV1.redeem()

Description

redeem() has 2 calls to forceApprove() to allow the vaults involved in the redemption to spend the tokens. These calls are not necessary because of a check if the caller == owner within the Openzeppelin ERC4626 implementation used by Morpho.

Source Code

<https://github.com/wrytlabs/usdu-core/blob/bf78b7dc8e148f0b805980f7b9e65a4e5d738d50/contracts/morpho/MorphoAdapterV1.sol#L169-L200>

```
169 function redeem(uint256 sharesStaked) external onlyCurator {
170     // reconcile, triggers `_accruedFeeAndAssets` in vault
171     _reconcile(totalAssets(), true);
172
173     // approve staked shares for redeem from staked vault
174     staked.forceApprove(address(staked), sharesStaked);
175     uint256 sharesCore = staked.redeem(sharesStaked, address(this), address(this));
176
177     // approve core shares for redeem from core vault
178     core.forceApprove(address(core), sharesCore);
179     uint256 amount = core.redeem(sharesCore, address(this), address(this));
180
181     // reduce minted amount
182     if (totalMinted >= amount) {
183         stable.burn(amount);
184         totalMinted -= amount;
185     } else {
186         // fallback, burn existing totalMinted if available
187         if (totalMinted > 0) {
188             stable.burn(totalMinted);
189             totalMinted = 0;
190         }
191
192         // fallback, distribute remaining balance
193         uint256 bal = stable.balanceOf(address(this));
194         if (bal > 0) {
195             _distribute(bal);
196         }
197     }
198
199     emit Redeem(amount, sharesCore, sharesStaked, totalMinted);
200 }
```

Suggestion

Remove these redundant calls to save some gas.

Resolution

The calls were removed with [2556f573318d943242fbf137b96b32691322fcf6](#).

Techniques

A comprehensive examination has been performed utilizing Manual Review and Static Analysis techniques. The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors
- Assessing the code base to ensure compliance with current best practices and industry standards
- Ensuring contract logic meets the specifications and intentions of the client
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders
- Thorough line-by-line manual review of the entire code base

Disclaimer

The smart contracts provided by the client for audit purposes have been thoroughly analysed in compliance with the global best practices to date w.r.t Cybersecurity vulnerabilities and issues in smart contract code, the details of which are enclosed in this report.

This report is not an endorsement or indictment of the project or team, and they do not in any way guarantee the security of the particular object in context. This report is not considered, and should not be interpreted as an influence, on the potential economics of the token, its sale or any other aspect of the project. Crypto assets/tokens are the results of the emerging blockchain technology in the domain of decentralized finance and they carry with them high levels of technical risk and uncertainty.

No report provides any warranty or representation to any third-Party in any respect, including regarding the bug-free nature of code, the business model or proprietors of any such business model, and the legal compliance of any such business. No third party should rely on the reports in any way, including for the purpose of making any decisions to buy or sell any token, product, service, or other assets. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project.

Smart contracts are deployed and executed on a blockchain. The platform, its programming language, and other software related to the smart contract can have its vulnerabilities that can lead to hacks. The Solidity language itself remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer or any other areas beyond Solidity that could present security risks.